

CS 4530: Fundamentals of Software Engineering

Module 4: Architecture of a Web Application

Adeel Bhutta and Mitch Wand
Khoury College of Computer Sciences

Learning Goals for this Lesson

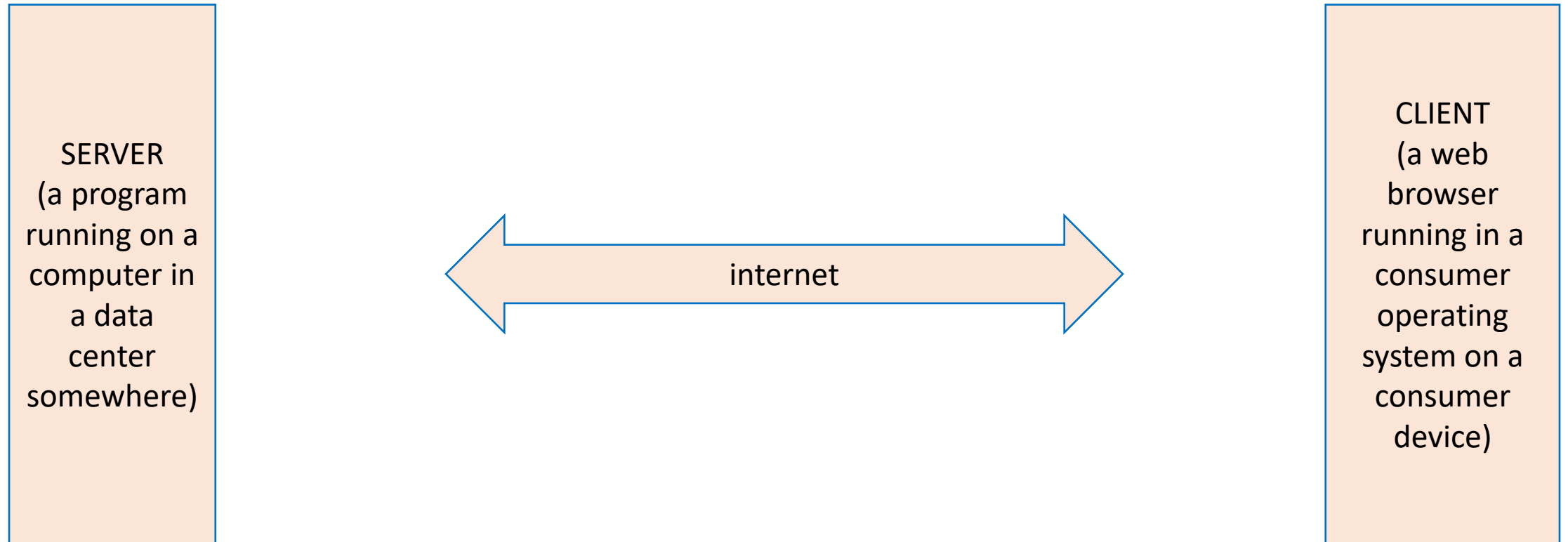
At the end of this lesson, you should be able to

- Explain the role of “client” and “server” in the context of web application programming
- Explain the role of HTTP endpoints and REST APIs versus WebSocket communication and event-driven messaging in a web application
- Describe the fundamental differences between the three layers of the controller, service, and repository layers in a C-S-R architecture
- Understand how the C-S-R architecture works in the context of a basic Express application
- Be able to answer an interview question about “business logic,” “horizontal and vertical scaling,” or “microservices”

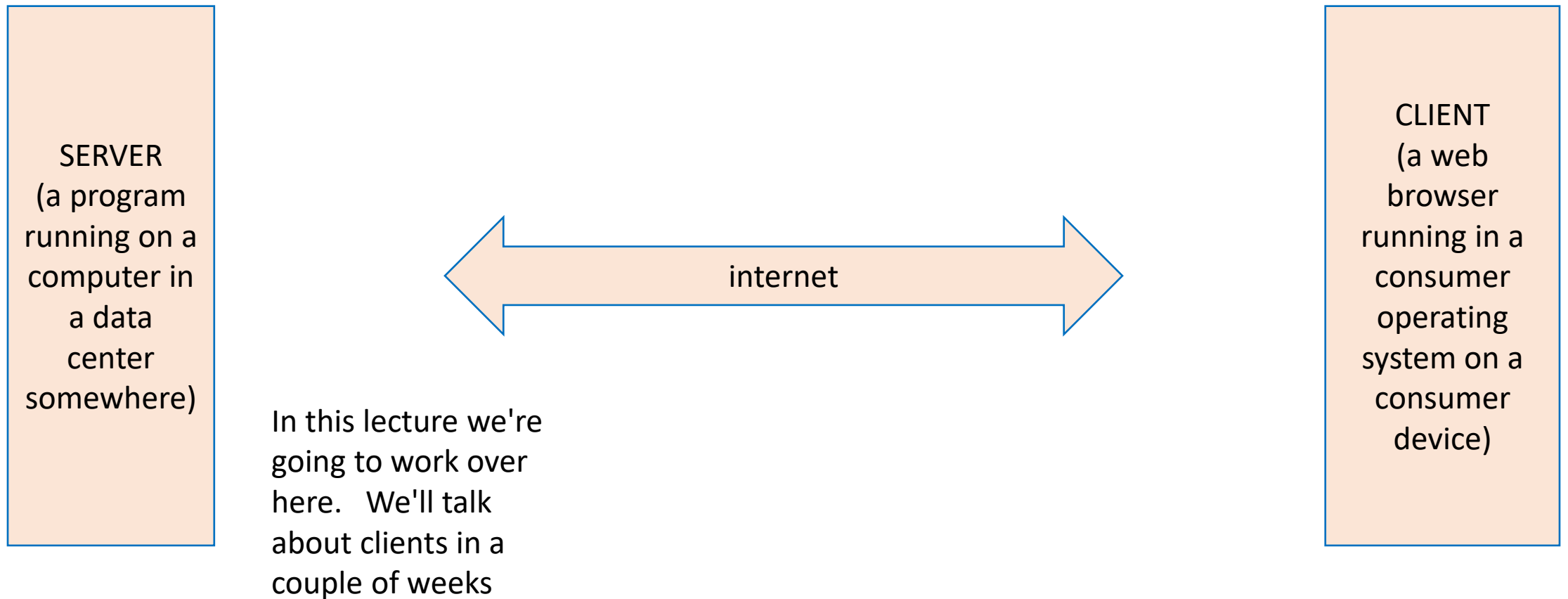
Web Applications are Distributed Systems

- Distributed systems are applications that perform computation across different computers
- Web applications are distributed systems *because*
 1. Users are literally in different places: you don't live in a data center
 2. Netflix won't fit entirely on one computer
- Distributed systems are hard!
 - Adopting well-understood patterns makes the task easier
 - We'll talk about one kind of pattern in this lecture
 - If you're Google, Netflix, or Amazon, the simple designs in this lecture won't serve your purposes

The simplest distributed system has a server and one or more clients



The server and the client communicate using an agreed-upon protocol

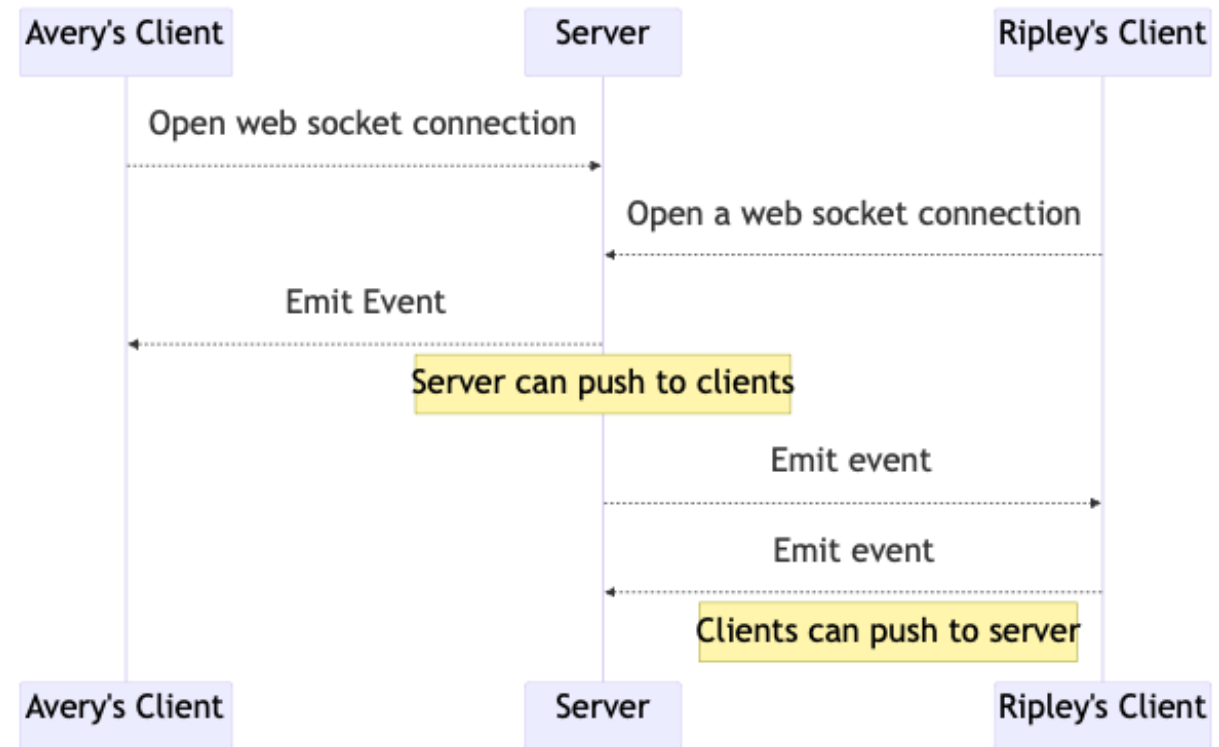


Examples of protocols

- Checking guests off a guest list
- Morse code
- Procedure calls
 - how are arguments passed to procedures?
 - how are results passed back to the caller?
- The system designer gets to choose the protocol, and then builds the programs to match

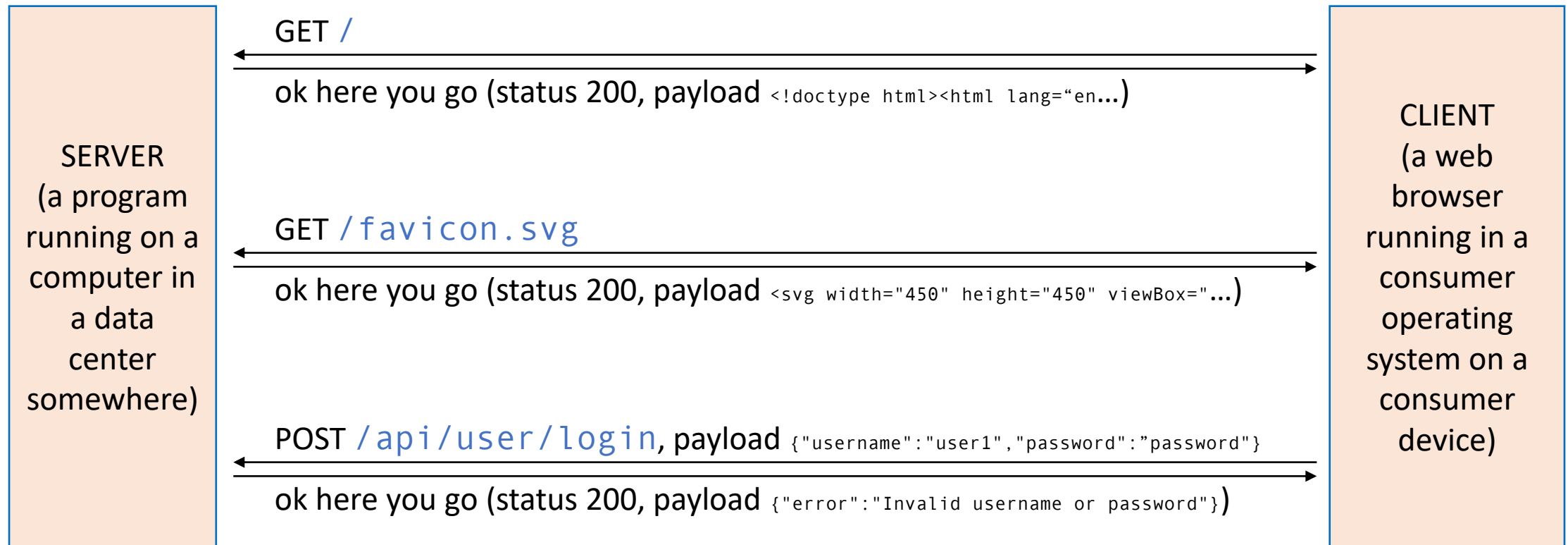
WebSockets is a useful protocol for event-driven applications

- A server can communicate with many clients
- A chat message is an “event” that the server wants everyone to know about.
- The web server is a *centralized* point that allows distributed clients to talk to each other.
 - Centralization is a cheat code for making distributed systems manageable



HTTP is a useful protocol for "function-call"-like patterns

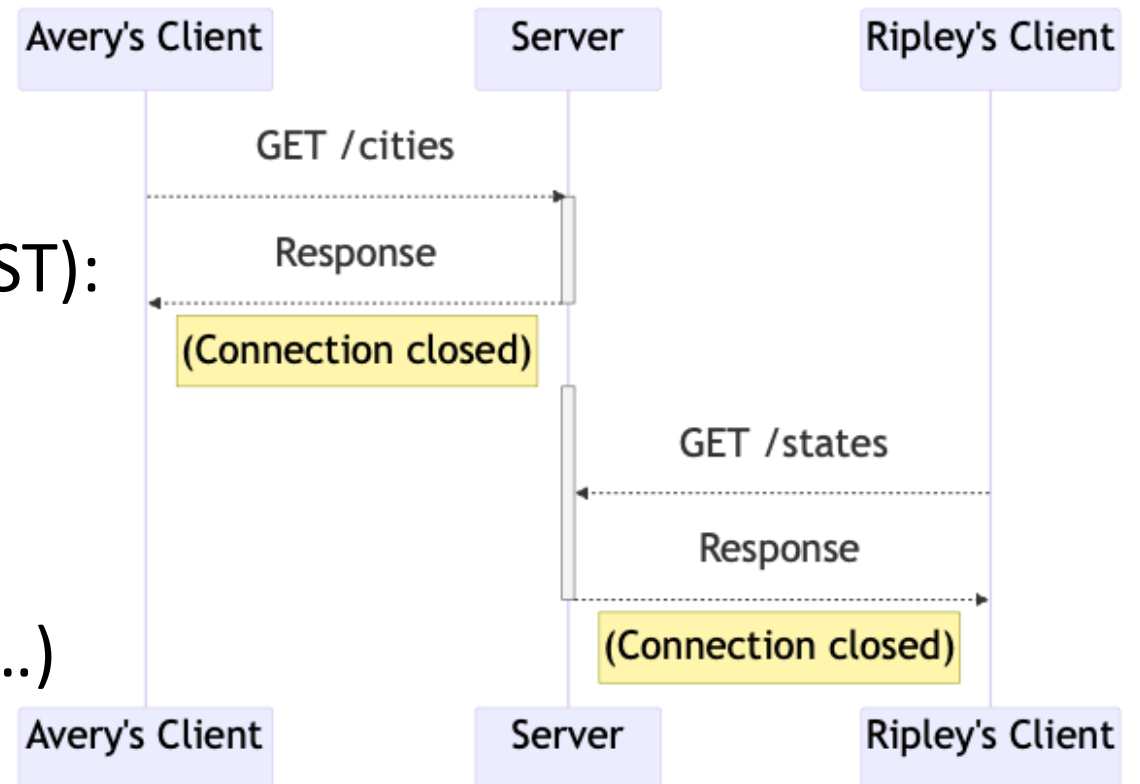
HTTP gathers messages in pairs: client sends, server responds



HTTP is a protocol useful for “Function-call-like” Design Patterns

HTTP supports common design patterns:

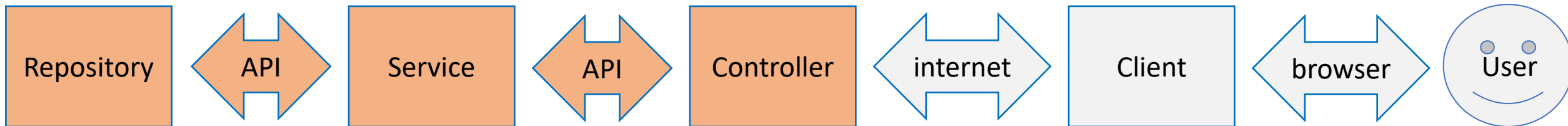
- *Remote Procedure Call (RPC)*:
one computer is calling a function on another computer
- *REpresentational State Transfer (REST)*:
strictly client/server, server doesn't maintain a session or remember specific clients
- A set of routes (like /cities, /states, ...) answering HTTP requests is a “**REST API**”



The Controller-Service-Repository Pattern

Three parts of a web server

- The Controller-Service-Repository design separates the server into three parts
 - The **controller** knows how we communicate with client, doesn't know how we store data
 - The **service** doesn't know how we connect to the client or store data, but does as much of the interesting work ("business logic") as possible without knowing these things
 - The **repository** is the only part stores information long-term



```
// app.ts
import express from 'express';
import * as controller from './controller.ts';
const app = express();
app.use(express.json());
app.post('/api/user/login', controller.controlLogin);
```

No, the Controller doesn't know about http; it's app.ts that knows about http. The controller just sends commands to the service.

```
// controller.ts
import type { Request, Response } from 'express';
let numLogins = 0;

export function controlLogin(request: Request, response: Response) {
  const { username, password } = request.body;
  if (username.toLowerCase() === 'user1' && password === 'sekret') {
    response.send({ success: true, numLogins: ++numLogins });
  } else {
    response.send({ error: 'Invalid username or password' });
  }
};
```

The top level of the app defines routes for the API endpoints, and delegates t

- The top level of the app defines the

The Controller (with Zod Validation)

```
// controller.ts
import type { Request, Response } from 'express';
import { z } from 'zod';

let numLogins = 0;
const zLoginReq = z.object({ username: z.string(), password: z.string() });
export function loginController(request: Request, response: Response) {
  const auth = zLoginReq.safeParse(request.body);
  if (!auth.success) {
    response.send({ error: 'Bad request' });
  } else if (auth.data.username.toLowerCase() === 'user1' && auth.data.password === 'secret') {
    numLogins += 1;
    response.send({ success: true, numLogins });
  } else {
    response.send({ error: 'Invalid username or password' });
  }
};
```

The Controller Relies on the Service Layer

```
// controller.ts
import type { Request, Response } from 'express';
import { z } from 'zod';
import { isAuthenticated, incrementLogins } from './service.ts';

const zLoginReq = z.object({ username: z.string(), password: z.string() });
export function controlLogin(request: Request, response: Response) {
  const auth = zLoginReq.safeParse(request.body);
  if (!auth.success) {
    response.send({ error: 'Bad request' });
  } else if (isAuthenticated(auth.data.username, auth.data.password)) {
    const numLogins = incrementLogins();
    response.send({ success: true, numLogins });
  } else {
    response.send({ error: 'Invalid username or password' });
  }
}
```

The controller should validate inputs

The controller uses the service layer to construct an appropriate response

The Service Layer Has Most of the Business Logic

```
// service.ts
```

```
export function isAuthenticated(username: string, password: string) {  
  return username.toLowerCase() === 'user1' && password === 'secret'  
}
```

```
let numLogins = 0;
```

```
export function incrementLogins() {  
  const oldNumLogins = numLogins;  
  numLogins += 1;  
  return oldNumLogins;  
}
```

- What user needs *aren't* being met here (and in the IP1 starter code?)
(Change password? Save numLogins when rebooting?)
- How can we do better?
(Add a database)

The Repository Layer Provides Persistence

- Logins should be cumulative even if we restart the server
- Adding users and changing passwords shouldn't necessarily require updating code
- Lots of ways to achieve this:
 - MongoDB
 - PostgreSQL
 - SQLite
 - A file on the hard drive

Back to the Transcript Server from the TDD Lecture

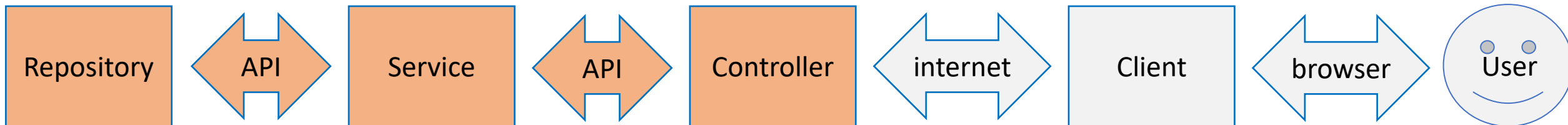
- Everything we saw from the transcript server is the **business logic** — a *boring-sounding* name that refers to most of the *interesting* stuff a web server does
 - “Is this HTTP request coming from a recognized user?” — not business logic
 - “Does this user have permission to access student records” — business logic!
 - “Do new grades go at the front or back of the list” — business logic!

Testing a Controller-Service-Repository Server

- Testing at the service layer tests the service+repository, testing by invoking API calls tests controller+repository+server
 - You can also test the controller level in isolation
 - What are the pros and cons of each?
- Sometimes we'll want to test the service layer and/or controller layer *without* the repository layer!
 - We'll come back to this.

Review: three parts of a web server

- The **repository** is the only part that stores information long-term
 - This is pretty much a synonym for “database”
- The **service** doesn't know how we connect to the client
 - HTTP? REST? WebSockets? The service shouldn't know!
 - Business logic lives here
- The **controller** doesn't know how we store data
 - Blissfully unaware of whether we're using a database or just memory (like in IP1)



4.3 Dealing with a Persistent Repository

The Problem

- Adding a persistent repository makes one big difference!
 - almost every action that reads or writes data is now *hundreds* of times slower, and involves reading to disk
 - this involves a relatively long delay, during which the CPU isn't doing anything useful
- JavaScript handles this with *asynchronous programming*.

Typescript uses the type system to distinguish long-running computations

- A long-running computation gets type **Promise<Grade>**, rather than **Grade**.

This is documented in the interfaces

```
export interface ITranscriptRepo {  
  addStudent(studentName: string): Promise<StudentID>;  
  getTranscript(id: StudentID): Promise<Transcript>;  
    // promise is if id invalid  
  deleteStudent(id: StudentID): Promise<void>;  
    // promise is rejected if id invalid  
  addGrade(id: StudentID, course: Course, courseGrade: CourseGrade):  
    Promise<void>;  
  getGrade(id: StudentID, course: Course): Promise<CourseGrade>;  
  nameToIDs(studentName: string): Promise<StudentID[]>;  
}
```

Use **await** to get the result of a promise (1)

```
describe('addStudent', () => {  
  it('should add a student to the database and return their id',  
    async () => {  
      expect(await repo.nameToIDs('blair')).toStrictEqual([]);  
      const id1 = await repo.addStudent('blair');  
      expect(await repo.nameToIDs('blair')).toStrictEqual([id1]);  
    });  
});
```

- repo.nameToIDs returns a promise that will compute a list of IDs
- here we need to see the actual list of IDs

Use **await** to get the result of a promise (2)

```
describe('addStudent', () => {  
  it('should add a student to the database and return their id',  
    async () => {  
      expect(await repo.nameToIDs('blair')).toStrictEqual([]);  
      const id1 = await repo.addStudent('blair');  
      expect(await repo.nameToIDs('blair')).toStrictEqual([id1]);  
    });  
});
```

- So long as we have a single-threaded app, this works just fine!
- If our app is juggling multiple threads, there are dangers--more in M08.

Any function that calls a long-running function is itself long-running

```
async function numberOfSharedNames(name: string): Promise<number>
{
    const ids = await db.nameToIDs(name);
    return ids.length
}
```

If every caller of an async function is itself async, where do you stop?

- when you reach the top level of a CLI
- when you reach the test harness (vite or jest)
- when you reach an express route (typically in the a controller)

Our tests also hide some technical debt

- We saw that we had to await the results of the calls
- But we also need to revisit what to do with errors
 - these are different in the async case
- But it may also be difficult or impossible to reset or undo changes in the database

```
let db: TranscriptService;  
beforeEach(() => {  
  db = new TranscriptDB();  
});
```

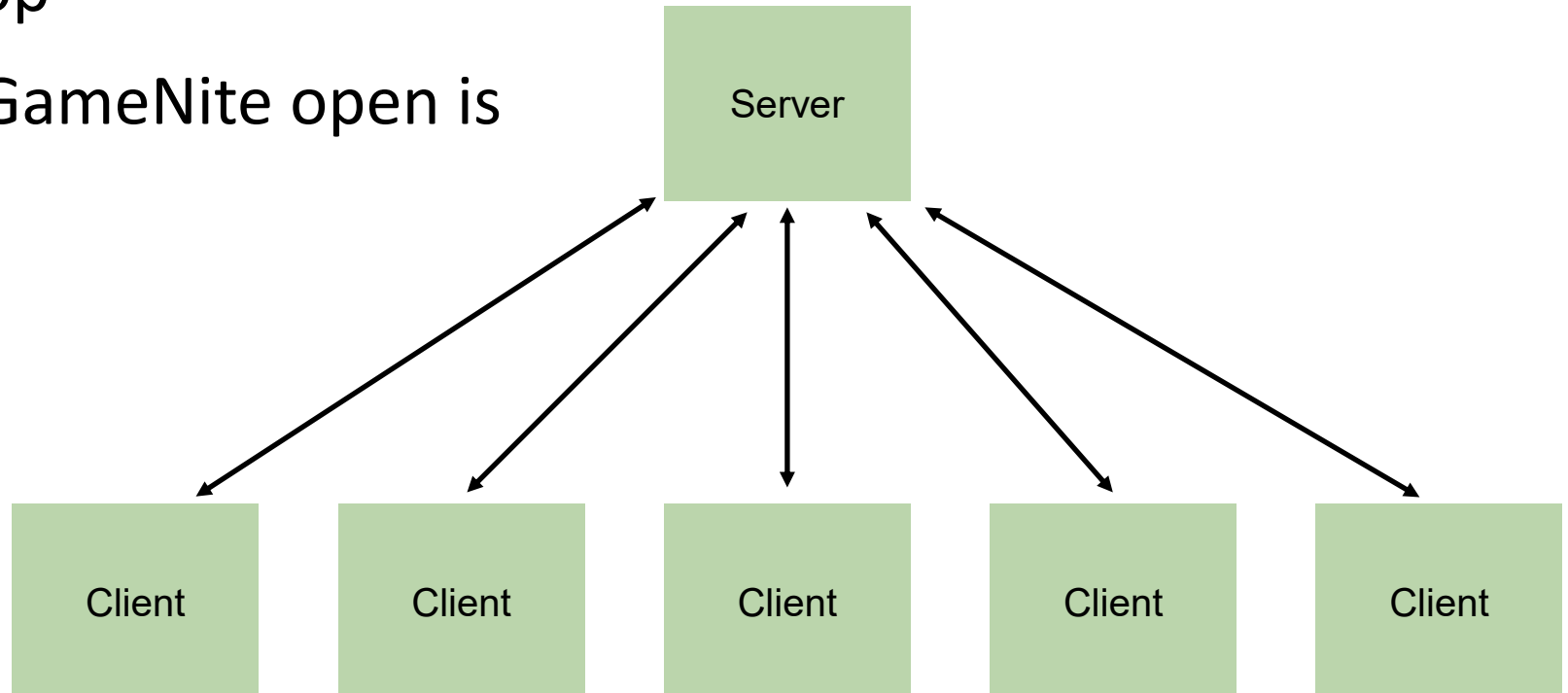
Review

At the end of this lesson, you should be able to

- Explain the role of “client” and “server” in the context of web application programming
- Explain the role of HTTP endpoints and REST APIs versus WebSocket communication and event-driven messaging in a web application
- Describe the fundamental differences between the three layers of the controller, service, and repository layers in a C-S-R architecture
- Understand how the C-S-R architecture works in the context of a basic Express application
- Understand the basic usage patterns for `async` and `await`.

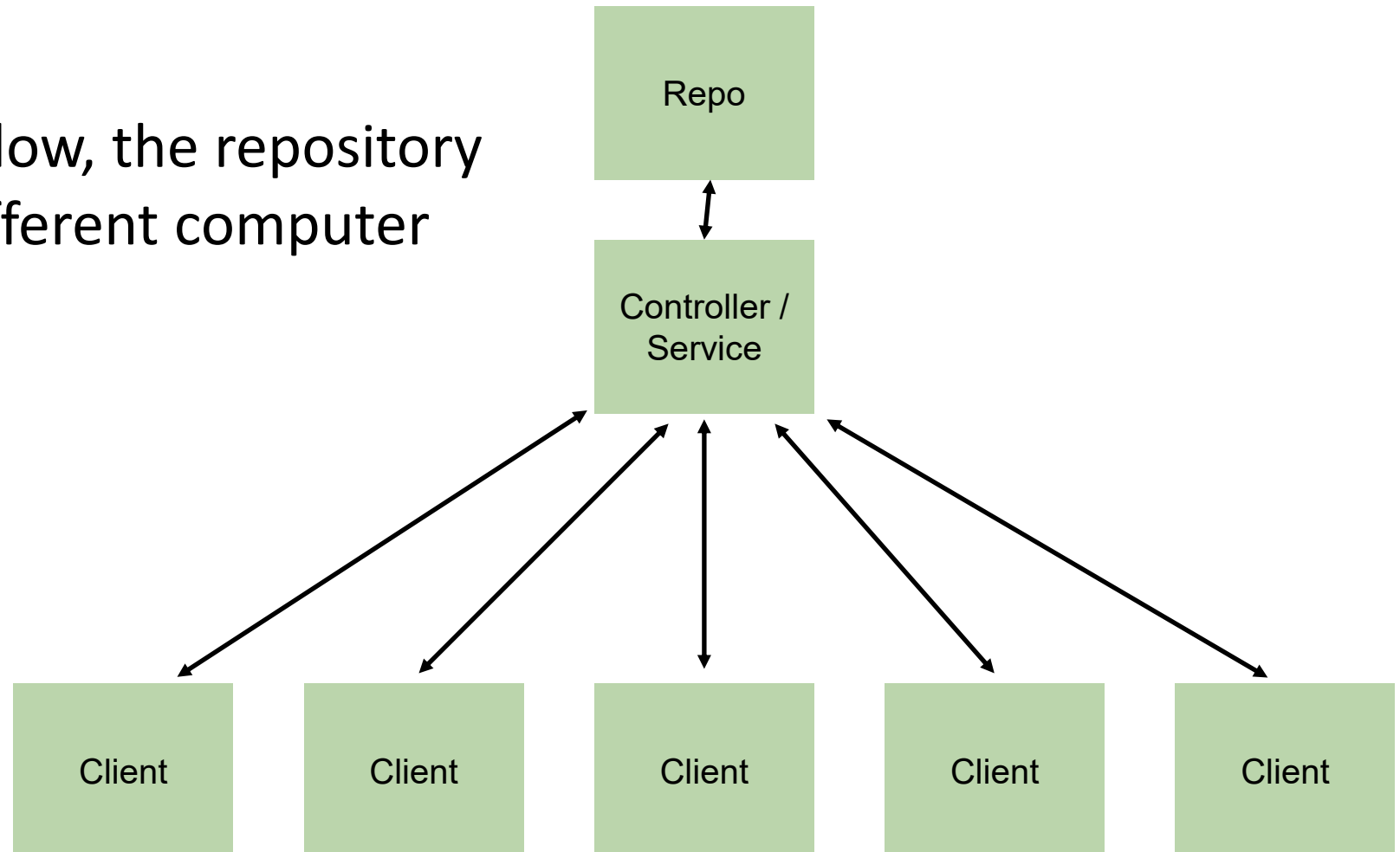
Web Applications: One Server, Many Clients

- We've mostly ignored so far that one server is supposed to connect to many clients
- This isn't always apparent when everything's running on your laptop
- Each web page with GameNite open is a different client



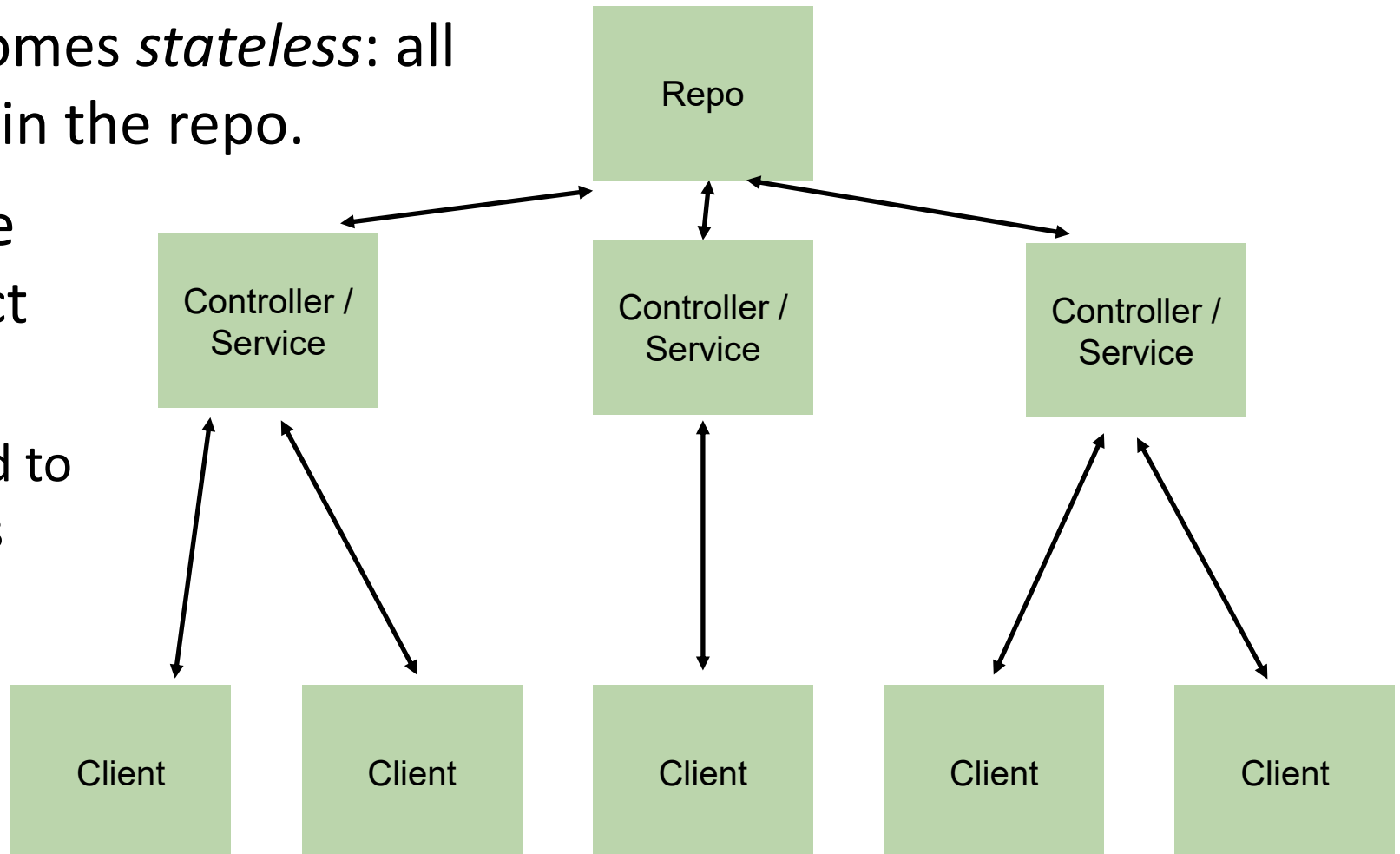
Scaling The C-S-R Architecture

- Web services often start on a single computer
- When that gets too slow, the repository is easy to put on a different computer



Horizontal Scaling

- By separating out the repository layer, the service layer becomes *stateless*: all the state it uses lives in the repo.
- Multiple copies of the controller can connect to multiple clients!
 - AWS will be delighted to help, only real limit is money

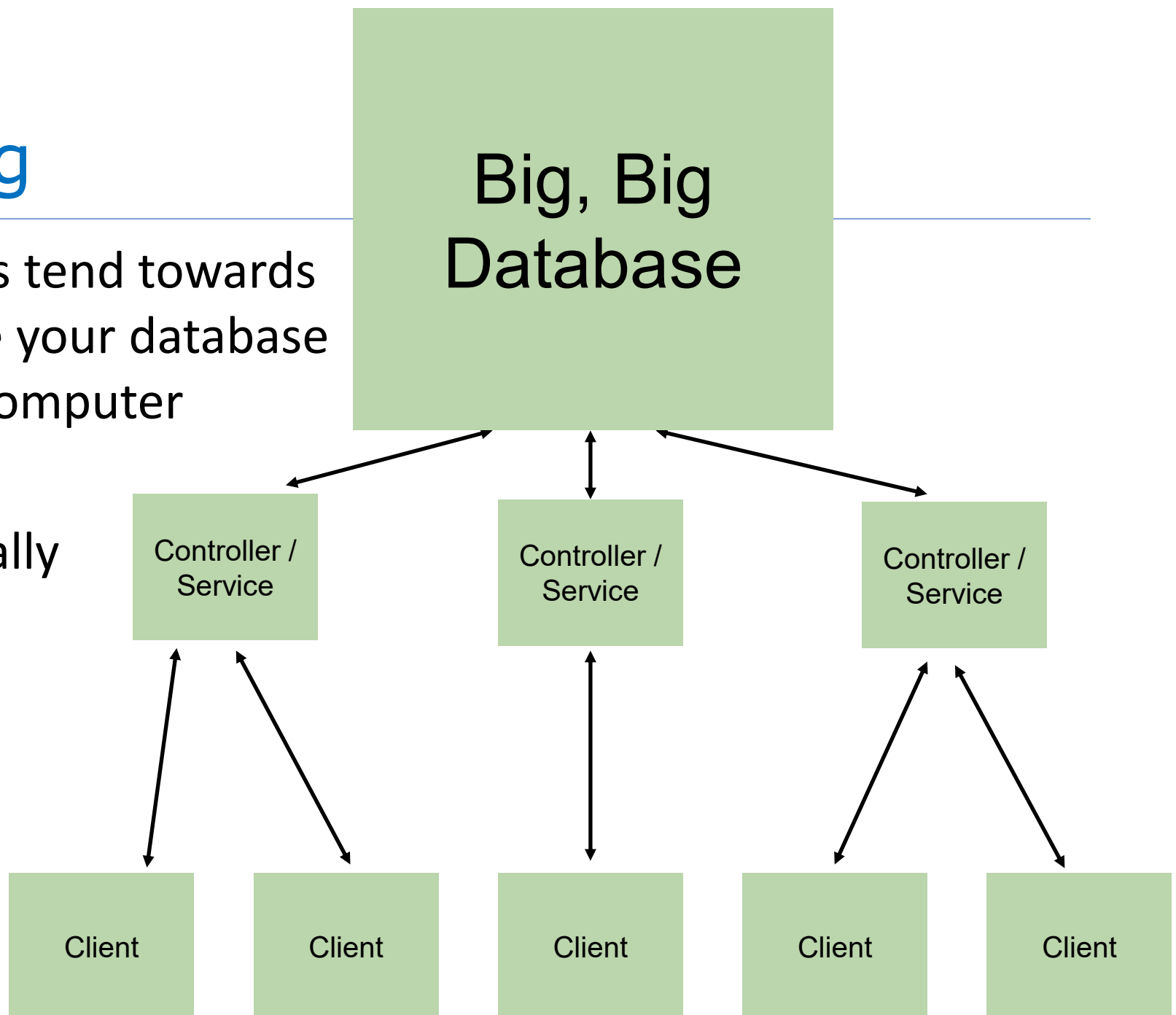


State and statelessness

- A web server or web service should be *stateless*
 - Every REST request should be indifferent to whether the node application has been *running* for several hours or five seconds
 - Also indifferent to whether other copies are communicating with different clients, as long as they're communicating with the same Repository
 - Our silly application, and the IP1 code, is *not* stateless (why?)
- If the web server is going to be stateless, and the web application has state, the server has to phone a friend (the Repository layer) to:
 - Access the filesystem
 - Query a database
- In C-S-R, the repository layer/database is the point of centralization
 - Centralization (& hierarchical centralization) is a cheat code for making distributed systems manageable

Vertical Scaling

- Centralized databases tend towards *vertical* scaling: move your database to a more powerful computer
- This has limits: the database will eventually become a bottleneck
- What to do?



Scaling Past the Database Bottleneck (1)

Maybe the problem is that you're occasionally doing big, expensive analysis on your database, like analyzing your sales data

- Database *read-only-replicas* are an easy solution here — seconds to minutes behind reality (and can add reliability in case of failure!)

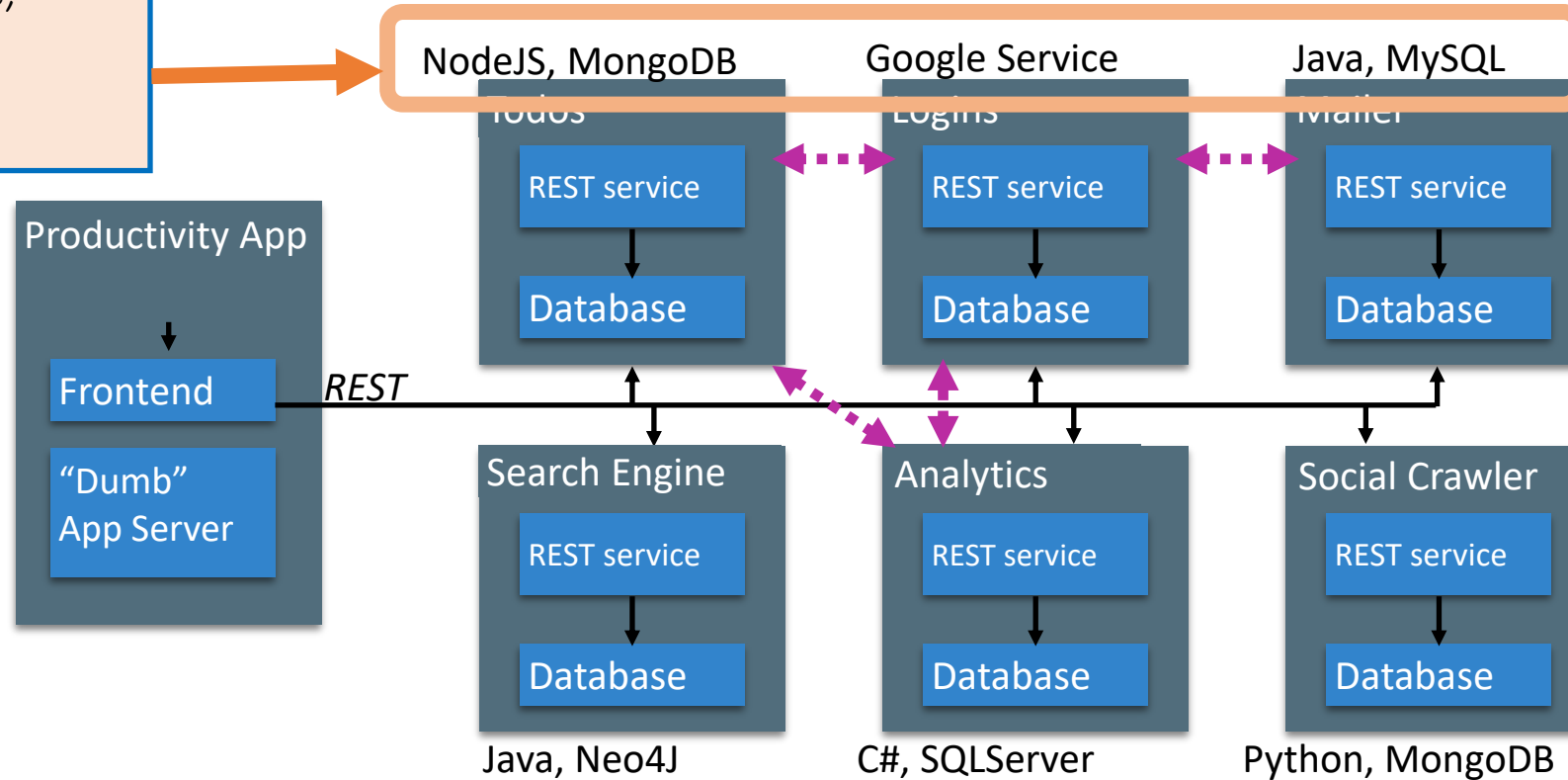
Scaling Past the Database Bottleneck (2)

Maybe you can identify parts of your data that are independent, and don't need to be synchronized or stored together

- Chat and game information in GameNite could live in separate places
- Games could have their business logic running on different servers, written in different programming languages, and accessed (by the server the client is connected to) through their own REST API!
 - This can help with need-more-CPU bottlenecks as well as need-a-beefier-database bottlenecks.
- This way lies microservices

Microservices

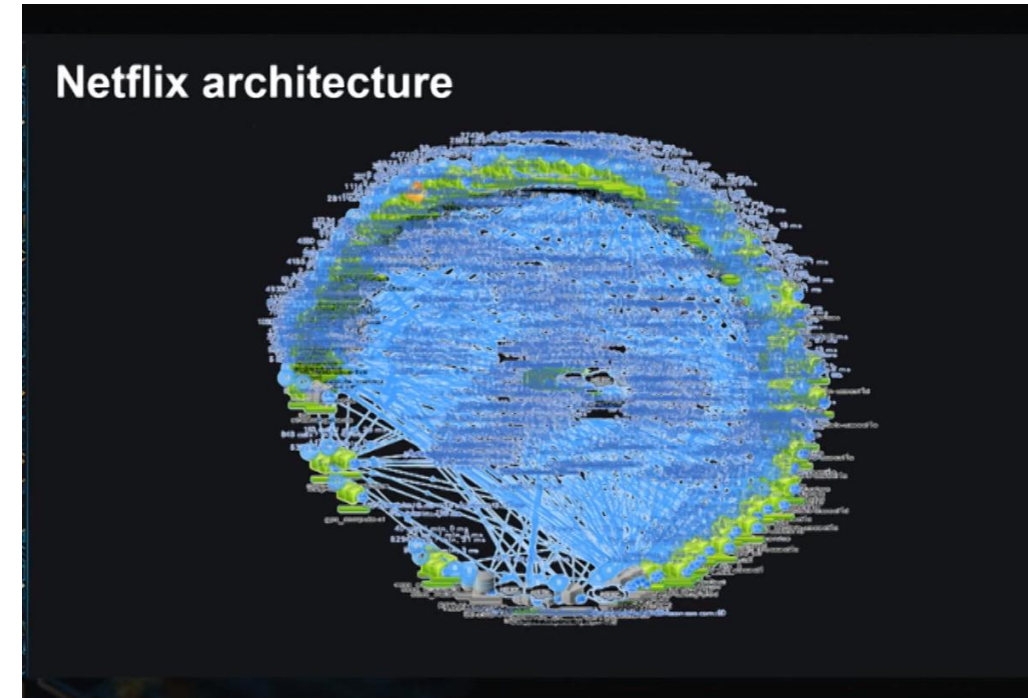
Different languages,
different operating
systems



Microservices

Netflix is the microservices darling

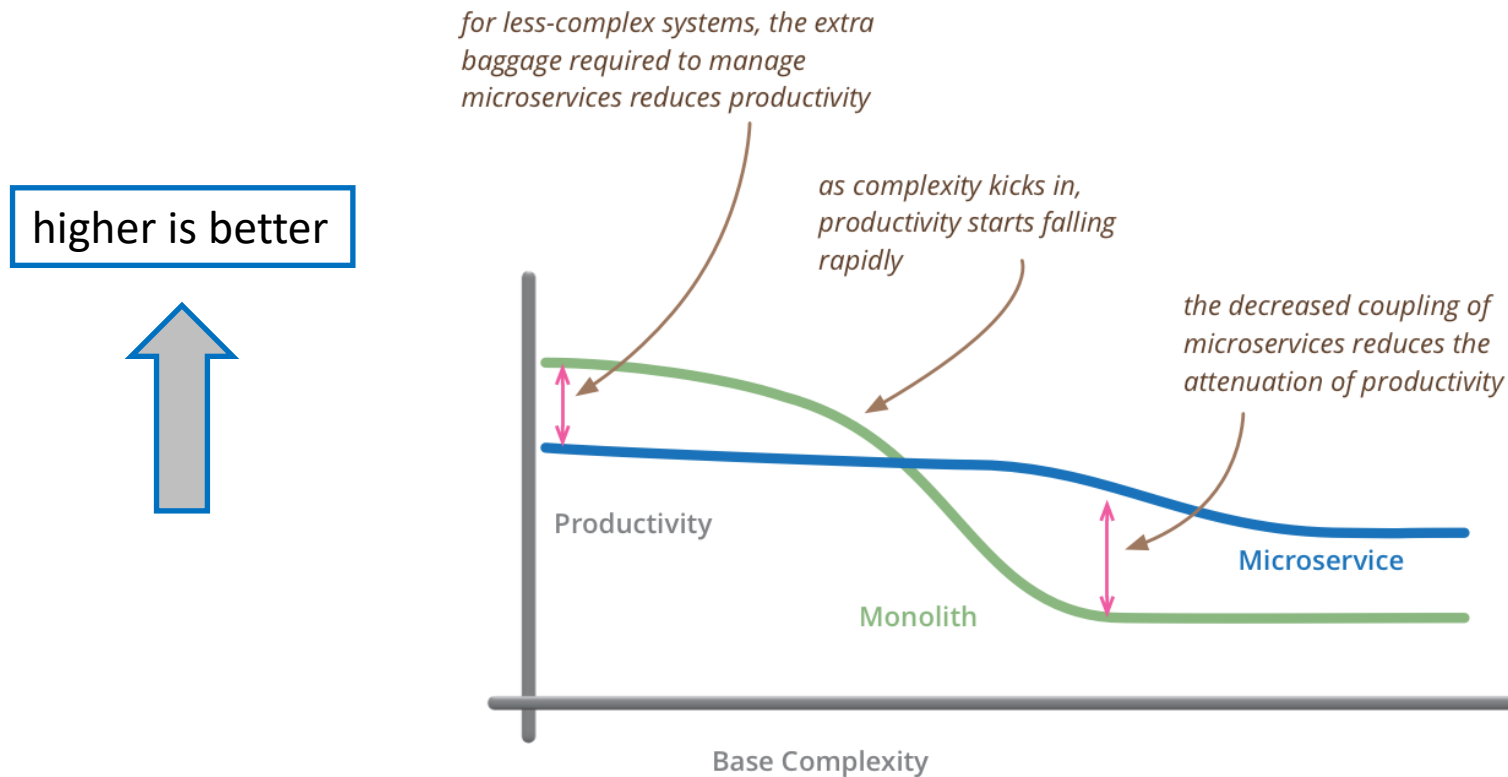
- 100s of microservices
- 1000s of daily production changes
- 10,000s of instances
- BUT:
- only 10s of operations engineers



<https://medium.com/refraction-tech-everything/how-netflix-works-the-hugely-simplified-complex-stuff-that-happens-every-time-you-hit-play-3a40c9be254b>

Microservices

The opposite of “microservices” is “monolith”



but remember the skill of the team will outweigh any monolith/microservice choice

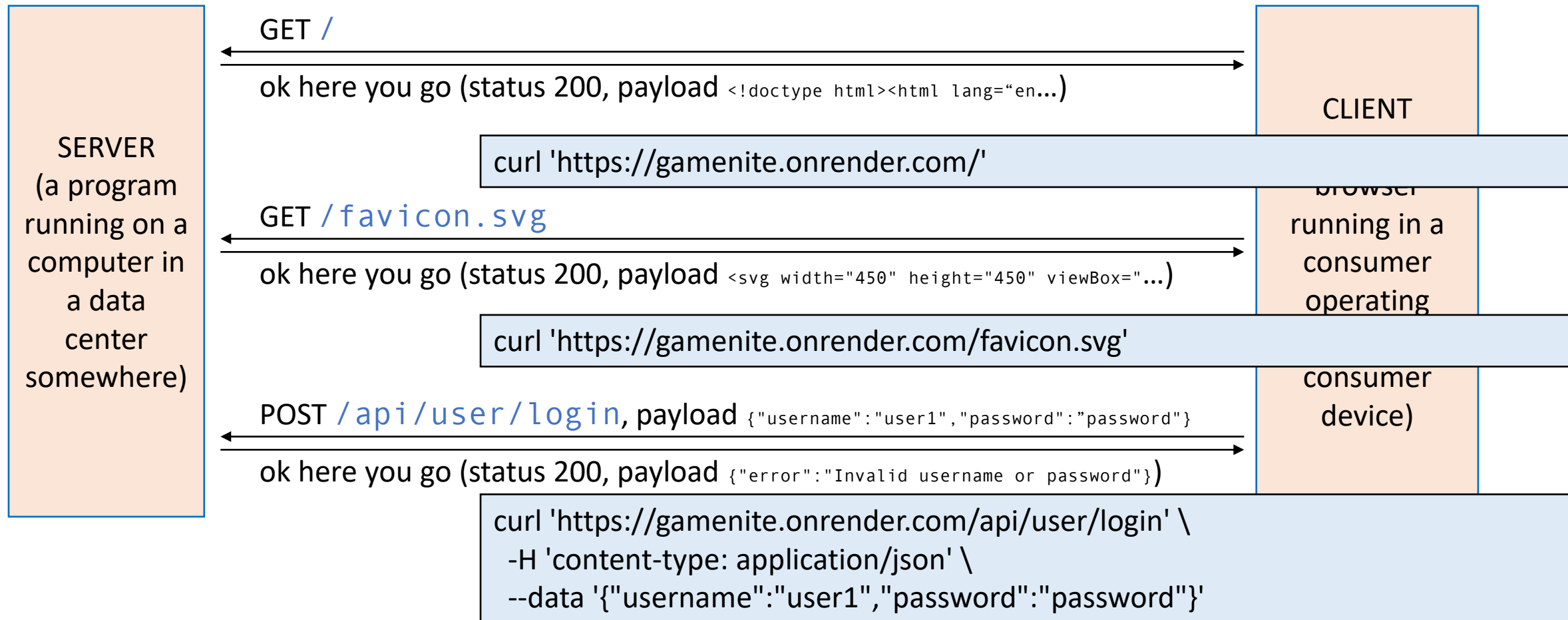
<https://martinfowler.com/microservices/>

GameNite is Monolithic

- GameNite is a monolithic application
- It's not perfect: there's probably a bit too much business logic in the controller layer (service layer doesn't quite do enough)
- You'll start IP2 with a more proper web app
 - MongoDB is the database used for repository layer, by way of a general-purpose adapter called Keyv
 - Changing the repository greatly changes the service layer
 - The controller doesn't change much (the controller is mostly unaware of the repository later)

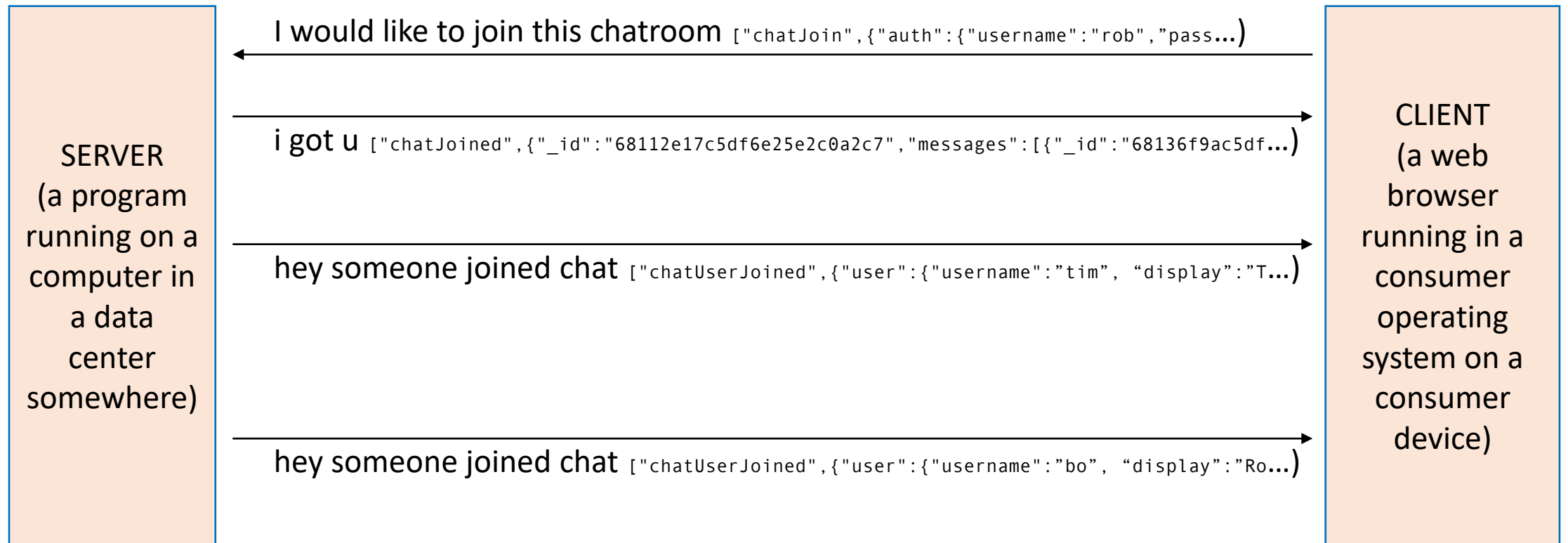
Application Layer Abstraction #1: HTTP

Remember: HTTP requests don't have to come from a browser!



Application Layer Abstraction #2: WebSockets

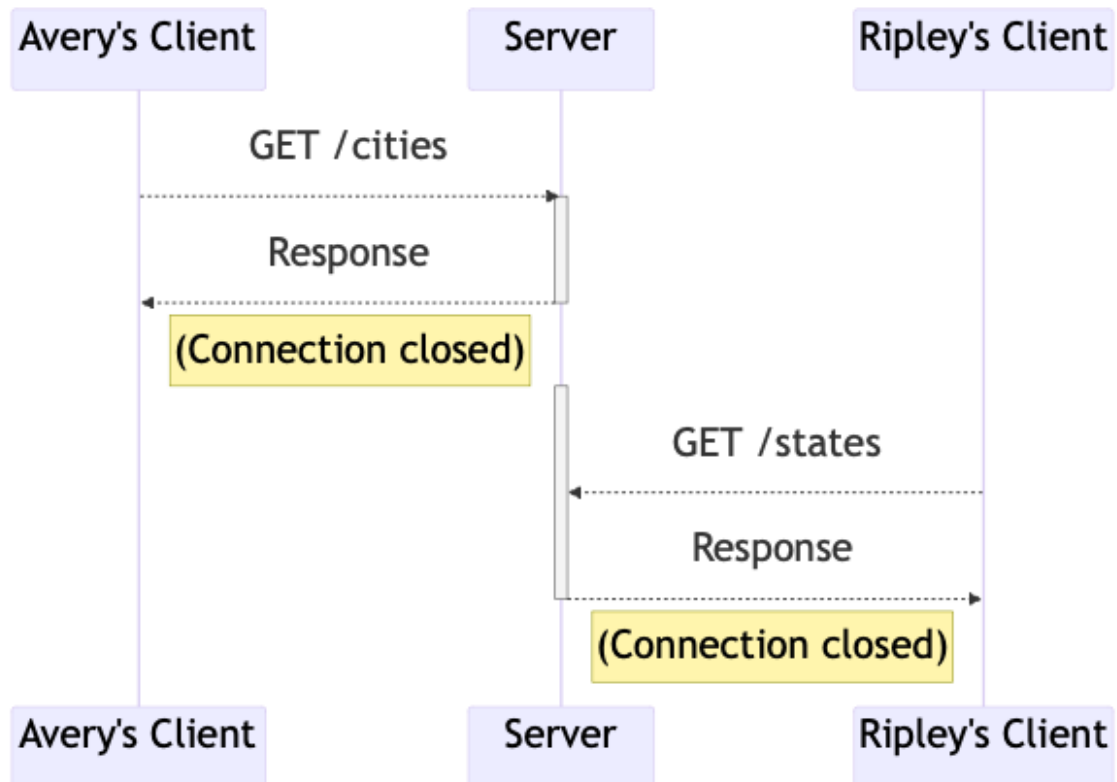
In a WebSockets session, client **or** server push messages (no response expected)



Application Layer Abstractions: HTTP Endpoints versus WebSockets

HTTP

(one-way, client initiates)



Web Sockets

(two-way after client opens socket)

